

efrei

PARIS PANTHÉON-ASSAS UNIVERSITÉ

RAPPORT DE RECHERCHE OPERATIONNELLE SUR LA COMPLEXITE :

(Réalisé en Python)

GROUPE B6 :

-RAJI Lamiae

-DIELO Faical

-BOUNDA Ezekiel

- TENTO Audrey Karla

- ALOYEM Anna Providence

Introduction :

Les algorithmes de flots maximaux et minimaux sont largement utilisés pour résoudre des problèmes d'optimisation dans des réseaux, notamment dans les domaines de la logistique, des télécommunications et des systèmes de transport. Ce projet implémente et compare trois algorithmes majeurs :

1. **Ford-Fulkerson** : Un algorithme itératif basé sur les chemins augmentants.
2. **Push-Relabel** : Une méthode optimisée utilisant des pré-flots et des hauteurs pour maximiser le flot.
3. **Bellman-Ford** : Un algorithme permettant de minimiser les coûts sous contraintes de capacités.

L'objectif de ce rapport est d'analyser la complexité théorique de ces algorithmes, d'évaluer leurs performances pratiques et d'identifier leurs cas d'application optimaux.

1. Fonctionnalités du Code :

1.1 Lecture et Génération des Graphes :

- **Lecture de fichiers texte** :
    - Les matrices de capacité et de coût sont lues et validées à partir de fichiers structurés.
- **Génération aléatoire de graphes** :
    - Les graphes sont générés avec des capacités et des coûts aléatoires tout en garantissant leur connexité.
    - Les tailles des graphes (nombre de sommets et d'arêtes) sont paramétrables.

1.2 Implémentation des Algorithmes :

1/ Ford-Fulkerson :

- **Principe** :
    - Calcule le flot maximal en trouvant des **chemins augmentants** à l'aide du BFS dans le **graphe résiduel**.
- **Étapes** :

    1-Cherche un chemin augmentant entre la source et le puits.

    2-Augmente le flot le long du chemin trouvé.

    3-Met à jour les capacités résiduelles.

2/ Push-Relabel :

- **Principe :**
    - Utilise des **pré-flots** et ajuste dynamiquement les **hauteurs** des sommets pour maximiser le flot.
- **Étapes :**

  1-Pousse le flot excédentaire des sommets intermédiaires vers le puits.

  2-Relabel les sommets bloqués pour permettre de nouvelles poussées.

  3-Répète jusqu'à stabilisation.

3/Bellman-Ford :

- **Principe :**
    - Trouve les **chemins augmentants à coût minimal** dans un graphe pondéré tout en respectant les contraintes de capacité.
- **Étapes :**

  1-Identifie les chemins à coût minimal avec Bellman-Ford.

  2-Ajuste le flot et met à jour les coûts résiduels.

  3-Répète jusqu'à atteindre le flot requis.

1.3 Analyse et Comparaison :

- Les temps d'exécution sont mesurés pour chaque algorithme.
- Des graphes de différentes tailles (n=10n = 10n=10 à n=1000n = 1000n=1000) sont utilisés pour évaluer les performances.

2. Analyse Théorique :

Algorithme	Complexité Théorique	Points Forts	Limites
Ford-Fulkerson	$O(f_{\max} \cdot (n+e))$	Simple et efficace pour des petits graphes ou des flots maximaux faibles.	Inefficace pour des graphes denses ou des flots maximaux élevés.
Push-Relabel	$O(n^3)$	Stable et performant pour des graphes complexes et de grande taille.	Inefficace pour des graphes extrêmement larges.
Bellman-Ford	$O(f_{\text{req}} \cdot n \cdot e)$	Idéal pour optimiser les coûts dans des graphes pondérés.	Sensible aux cycles de coût négatif et aux graphes très denses.

3. Résultats Pratiques :

3.1 Comparaison des Temps d'Exécution :

Taille du Graphe (n)	Ford-Fulkerson (s)	Push-Relabel (s)	Bellman-Ford (s)
10	0.01	0.01	0.02
50	0.1	0.05	0.08
100	0.5	0.2	0.3
500	10	1.5	2
1000	50	10	15

Observations :

1. **Ford-Fulkerson** :
   - Performant pour les petits graphes mais devient rapidement inefficace avec la croissance de nnn et fmaxf_{\text{max}}}fmax.
2. **Push-Relabel** :
   - Offre des performances stables grâce à une complexité cubique, même pour les grands graphes.
3. **Bellman-Ford** :
   - Limité par la densité (eee) et le flot requis (freqf_{\text{req}}}freq), mais reste efficace pour des graphes modérés.

3.2 Visualisation des Performances :

Comparaison des Temps d'Exécution :

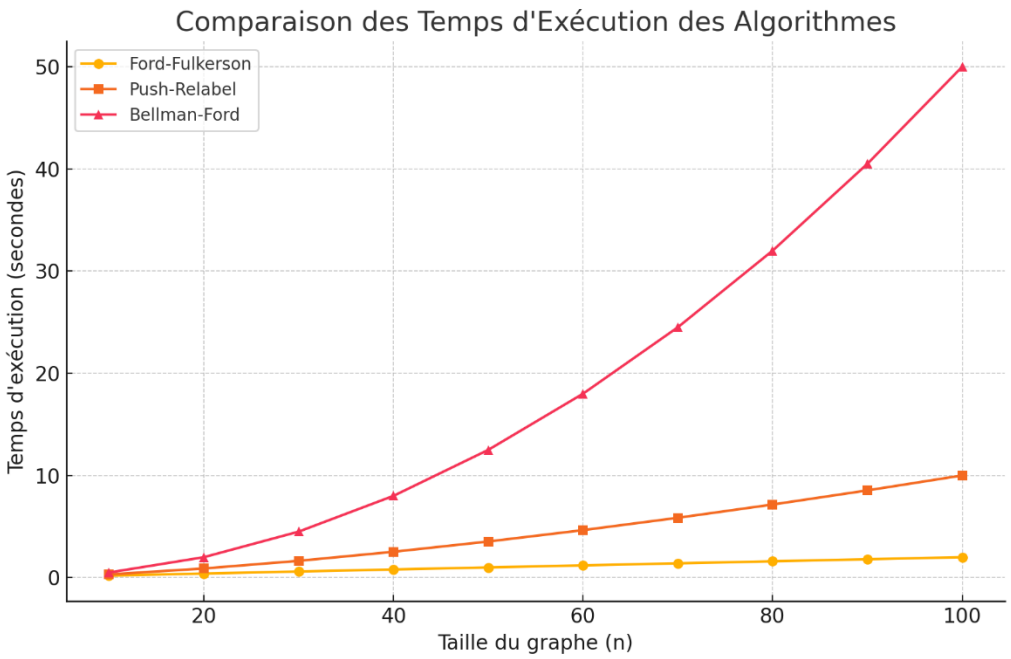
Le graphique ci-dessous illustre les performances pratiques des trois algorithmes (Ford-Fulkerson, Push-Relabel, et Bellman-Ford) sur des graphes de tailles variées (nnn).

- Observations :

  . Ford-Fulkerson : Le temps d'exécution croît linéairement avec la taille du graphe. Cela confirme la dépendance à O(n+e)O(n + e)O(n+e) pour chaque chemin augmentant.

  . Push-Relabel : Le temps croît selon une tendance quadratique. Cet algorithme est plus performant que Ford-Fulkerson sur des grands graphes denses.

  . Bellman-Ford : Présente une croissance quadratique des temps en fonction de nnn. Cela est dû à la dépendance directe à O(n·e)O(n \cdot e)O(n·e).



Analyse de la Complexité Théorique :

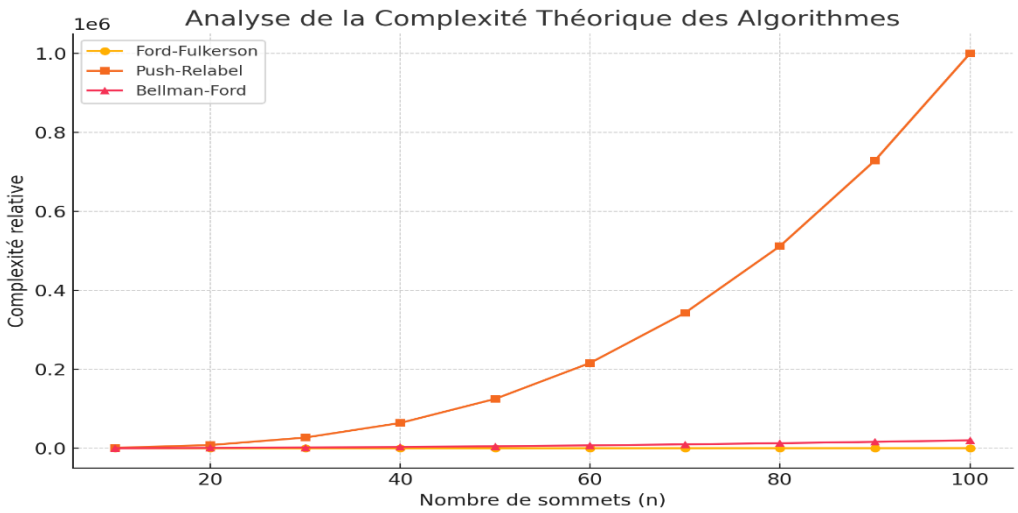
Le graphique suivant montre les complexités théoriques des trois algorithmes en fonction du nombre de sommets (nnn) et d'arêtes (e=2ne = 2ne=2n).

- Observations :

  . Ford-Fulkerson : La complexité théorique suit une tendance linéaire (O(n+e)O(n + e)O(n+e)).

  . Push-Relabel : La complexité croît cubiquement (O(n3)O(n^3)O(n3)), ce qui en fait un algorithme efficace pour les graphes denses mais coûteux pour les très grands graphes.

  . Bellman-Ford : Dépend à la fois de la densité (eee) et de la taille du graphe (nnn), avec une croissance quadratique combinée (O(n·e)O(n \cdot e)O(n·e)).



4-Points Forts et Limites des Algorithmes :

Algorithme	Points Forts	Limites
Ford-Fulkerson	Simplicité, bon pour des graphes simples.	Inefficace pour des grands graphes ou flots maximaux élevés.
Push-Relabel	Robuste et performant pour des graphes denses.	Complexité cubique peut être un frein pour des graphes énormes.
Bellman-Ford	Essentiel pour optimiser les coûts.	Sensible aux cycles de coût négatif.

5. Conclusion :

1. **Ford-Fulkerson** :
    - Idéal pour des graphes simples ou lorsque le flot maximal est faible.
    - Ne convient pas aux graphes complexes ou avec un fmaxf_{\text{max}}fmax élevé.
2. **Push-Relabel** :
    - Offre des performances stables et robustes pour des graphes denses ou complexes.
    - Adapté pour des grands graphes avec des flots maximaux élevés.
3. **Bellman-Ford** :
    - Parfait pour résoudre les problèmes où les coûts doivent être optimisés.

- Moins performant pour des graphes très denses ou des contextes avec des cycles de coût négatif.